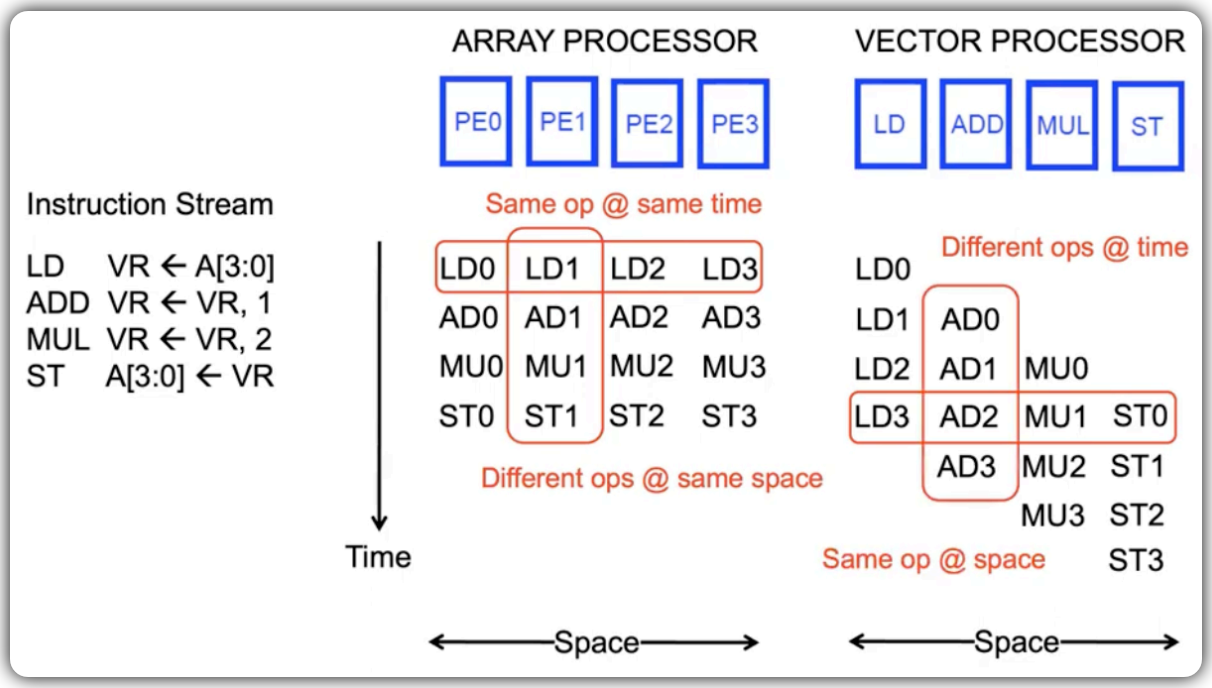
🎧 chapter 01: SIMD Processing and GPUs

Understanding heterogeneous systems is crucial in modern computing. This lecture provides an introduction to the architecture and programming model of **SIMD** processors and **GPUs** , focusing on their distinct capabilities. By exploring the classifications of processors and the principles of **array** and **vector processors** , viewers will gain insights into optimizing performance through **vectorization** and memory management techniques. The discussion also highlights the significance of **banked memory** and methods to prevent bank conflicts , essential for effective processing in heterogeneous environments. Overall, this content demystifies the complexities of GPU programming and illustrates vital concepts for unleashing their power in applications.

1. 🚀 Introduction to SIMD Processors and Their Architecture

- Current computing systems frequently incorporate **heterogeneous devices** like CPU cores, GPU cores, and FPGAs, which can exchange data efficiently via coherent interconnects.
    - **Heterogeneous devices**: Different computer parts working together as a team
        - Each component has a specialized role
    - **Key Components**:
        - **CPU cores**: The brain of the computer
        - **GPU cores**: Specialized for graphics and video processing
        - **FPGAs**: Flexible components that can adapt to different roles
    - **Communication**:
        - Uses **coherent interconnects**
        - Enables smooth communication between components
        - Acts like a common language for all devices
    - **Benefits**:
        - **Enhanced efficiency**: Each device handles its specialized tasks
        - **Improved performance**: System runs more smoothly through task distribution
        - **Optimized processing**: Components focus on what they do best

- The lecture introduces SIMD processors, where one instruction stream handles multiple data elements, exemplified by **array** and **vector processors** .
    - **Time-space duality**
        - **Array processor**: Instruction operates on multiple data elements at the same time using different spaces (PEs)
        - **Vector processor**: Instruction operates on multiple data elements in consecutive time steps using the same space (PE)
    - **diagram**



- Array processors handle multiple data elements simultaneously in different spaces, while vector processor s do so in consecutive time steps but the same space.
- GPU architecture often relies on SIMD processing, where multiple processing elements execute operations either temporally or spatially.
- Key processes in SIMD architectures such as instruction loading and arithmetic operations are demonstrated, highlighting the differences between array and vector processor paradigms.
    - **Vector Processor Definition**
        - A processor that operates on vectors rather than scalar (single data) values
    - **Basic Requirements**
        - Need to load/store vectors in **vector registers** (contain vectors)

    - Need to operate on vectors of different lengths using **vector length register (VLEN)**
    - Elements of a vector might be stored apart from each other in memory using **vector stride register (VSTR)**
        - **Stride**: Distance in memory between two elements of a vector

---

2. 🚀 Understanding SIMD Processor Architecture

🔗

- The **vector register's** size is determined by the product of n (number of elements) and m (bit size per element) and can hold n m-bit values.
- **Control registers** like vector length, vector stride , and vector mask help manage SIMD operations, with the mask enabling conditional operations .
    - **Vector length** register tells the system how many data pieces should be processed at once. Imagine it's like setting the number of cookies you want to bake on a tray at one time.
    - **Vector stride** helps in deciding the distance or gap between pieces of data being processed, kind of like skipping every other step when you climb stairs, so you touch every second stair.
    - **Vector mask** is used for deciding which particular data pieces should be included in the operation. It's like having a checklist that tells which cookies on the tray should get chocolate chips and which should not.
- **Banked memory** is used to improve memory throughput, allowing independent access to multiple memory banks in parallel.
- **Vectorizable code, such as independent loop iterations**, allows conversion from sequential to SIMD programs, optimizing performance.
- Using banked memory and pipeline architecture, vector processor s can hide latency by sustaining an access throughput of one element per cycle.

---

3. 🚀 Optimization and Execution in SIMD and GPUs

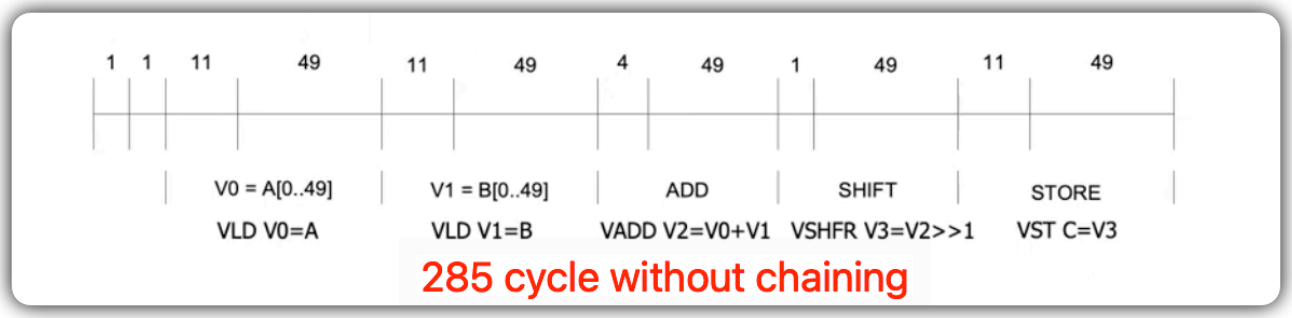
🔗

- **Chaining** can optimize performance by forwarding data from one vector unit to another, reducing cycle execution from 285 to 182.
- **Increasing memory ports** allows simultaneous data loads and stores, optimizing further to only 79 cycles.
- **Vector masks** are used in SIMD processors for predicated execution, allowing conditional operations based on specific criteria.
- **Bank conflicts** occur when memory stride s are not relatively prime; this can be minimized by increasing banks or ports.

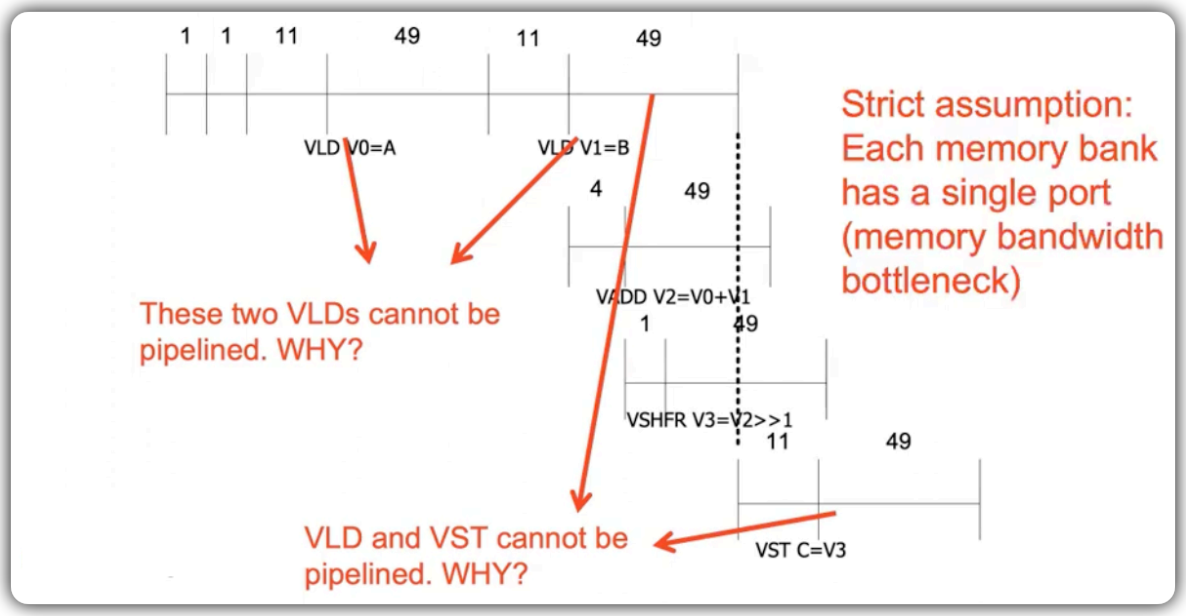
- GPUs use **scalar threads** in programming models, executing them as SIMD instructions in hardware for parallel processing.

3.1. Optimizing Vector Operations Through Chaining and Increased Memory Ports

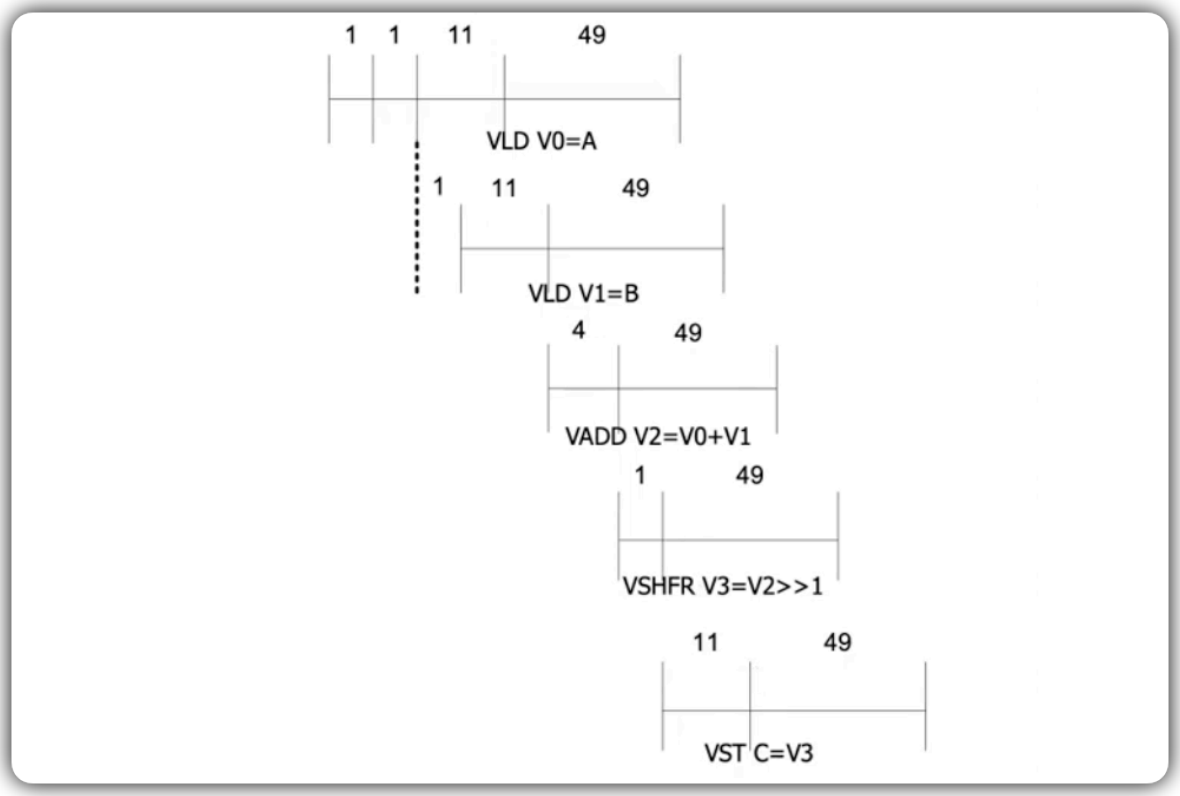
- A sequence of vector operations initially takes 285 cycles, highlighting potential for optimization.



- **Chaining** can reduce cycle counts by forwarding data between vector functional units, dropping cycle usage to 182.



- Further optimization is possible by performing concurrent loads and stores, requiring **increased memory ports**.



- By adopting two load ports and one store port per memory bank, loads and stores can overlap for efficiency.
- The optimized process reduces total cycles to 79, achieving a **19 times** performance improvement compared to sequential execution.

3.2. Masked Operations and Predicated Execution in SIMD

- **Vector masks** in SIMD processors facilitate masked operations, which are crucial when executing conditional statements.
- **Masked operations determine whether to execute an operation on a vector register, depending on whether conditions are satisfied or not.**
- An example involves checking if an array element is zero; if not zero, a multiplication occurs, whereas it is skipped if zero, since multiplying by zero gives zero.
- A **vector mask** is generated by evaluating conditions (predicates), producing a mask bit for each element. This bit determines the execution of operations.
- This mechanism is termed **predicated execution** , with operations contingent on mask bits, enabling selective execution in vectorized loops.

3.3. Key Considerations for Memory Access in SIMD Processors

- **Stride** and **banking** are significant issues to consider for memory access in SIMD processors.

- When a stride is one, accesses can be made to all banks simultaneously, avoiding conflicts.
- Bank conflicts arise when the number of banks and stride are not relatively prime, causing multiple requests to the same bank.
- In matrix operations, stride s not relatively prime to bank numbers can cause conflicts, especially when accessing rows by matrix A and columns by matrix B.
- **Solutions to minimize bank conflicts include increasing the number of banks or ports, changing data layout, and using randomized mapping techniques.**

Explanation:

In GPU memory architecture, **bank conflicts** occur when multiple threads in a warp (group of threads executed in parallel) try to access memory locations mapped to the same memory bank simultaneously. Memory banks are subdivisions of shared memory, allowing multiple threads to access shared memory in parallel.

Each bank can service only one request at a time. If two or more threads access the same bank simultaneously, a **bank conflict** happens, resulting in serialized access, which slows down performance.

Relationship to Banks and Strides

When threads access memory with a fixed **stride** (the offset between consecutive accesses), the stride determines how the memory locations map to the banks. Bank conflicts occur if the **stride** and the **number of banks** are **not relatively prime** (i.e., they have a common divisor greater than 1).

- If the **stride** and **number of banks** are relatively prime, memory accesses are evenly distributed across banks, avoiding conflicts.
- If they are not relatively prime, certain banks are accessed more frequently, leading to conflicts.

Example:

Assumptions:

- **Number of banks:** 4
- **Stride:** 2
- **Memory access pattern:** Each thread accesses an array index: `i * stride`.

Access Mapping:

1. Bank mapping is determined by the formula:

$$\text{Bank index} = (\text{Memory address}) \mod \text{Number of banks}$$

2. Consider 4 threads (`T0`, `T1`, `T2`, `T3`) with the following memory accesses:
   - **Thread 0 (T0):** Accesses index $0 \times 2 = 0$
   - **Thread 1 (T1):** Accesses index $1 \times 2 = 2$
   - **Thread 2 (T2):** Accesses index $2 \times 2 = 4$
   - **Thread 3 (T3):** Accesses index $3 \times 2 = 6$
3. Compute bank indices:

$$\text{Bank index for each thread} = (\text{Index}) \mod 4$$

- **T0:** $0 \mod 4 = 0$
- **T1:** $2 \mod 4 = 2$
- **T2:** $4 \mod 4 = 0$
- **T3:** $6 \mod 4 = 2$

4. Result: Threads `T0` and `T2` access **Bank 0**, while `T1` and `T3` access **Bank 2**. This creates **bank conflicts** because two threads try to access the same bank simultaneously.

No Conflicts (Relatively Prime):

If the stride were relatively prime to the number of banks, say **stride = 3**, the memory accesses would distribute across banks without conflicts:

- **T0:** $0 \mod 4 = 0$
- **T1:** $3 \mod 4 = 3$
- **T2:** $6 \mod 4 = 2$
- **T3:** $9 \mod 4 = 1$

Here, all threads access different banks: **0, 3, 2, 1**. No conflicts occur, and performance is optimal.

Key Takeaway:

Bank conflicts can be avoided by ensuring the stride is relatively prime to the number of banks. If not, accesses overlap on the same banks, leading to serialization and degraded performance.

3.4. Evolution of SIMD Extensions and Their Role in Modern Computing

- **SIMD extensions** began with MMX instructions in x86 processors, supporting 64-bit integer operations.
- **Successive improvements** led to Streaming SIMD Extensions (SSE), supporting 128-bit integers and floating-point operations, and later advancements to AVX with 256-bit and 512-bit computations.
- **Modern GPUs** , notably the A100, exemplify SIMD architectures with numerous parallel cores and specialized units for tasks like deep learning.
- **Shared memory in GPUs** is banked, allowing simultaneous access crucial for SIMD operations, though programming uses scalar threads instead of SIMD instructions.
- **Programming vs. execution models** are contrasted: programming deals with writing code, often in scalar threads , while execution models involve hardware execution, bundling

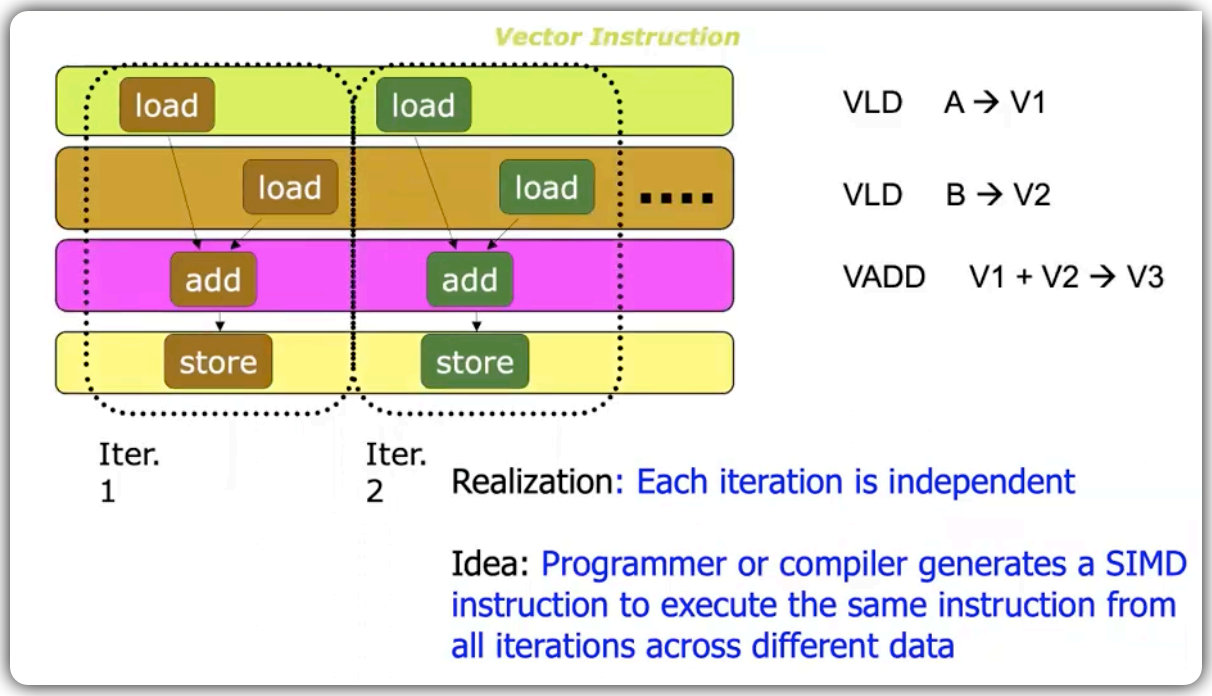
threads into SIMD-like units.

---

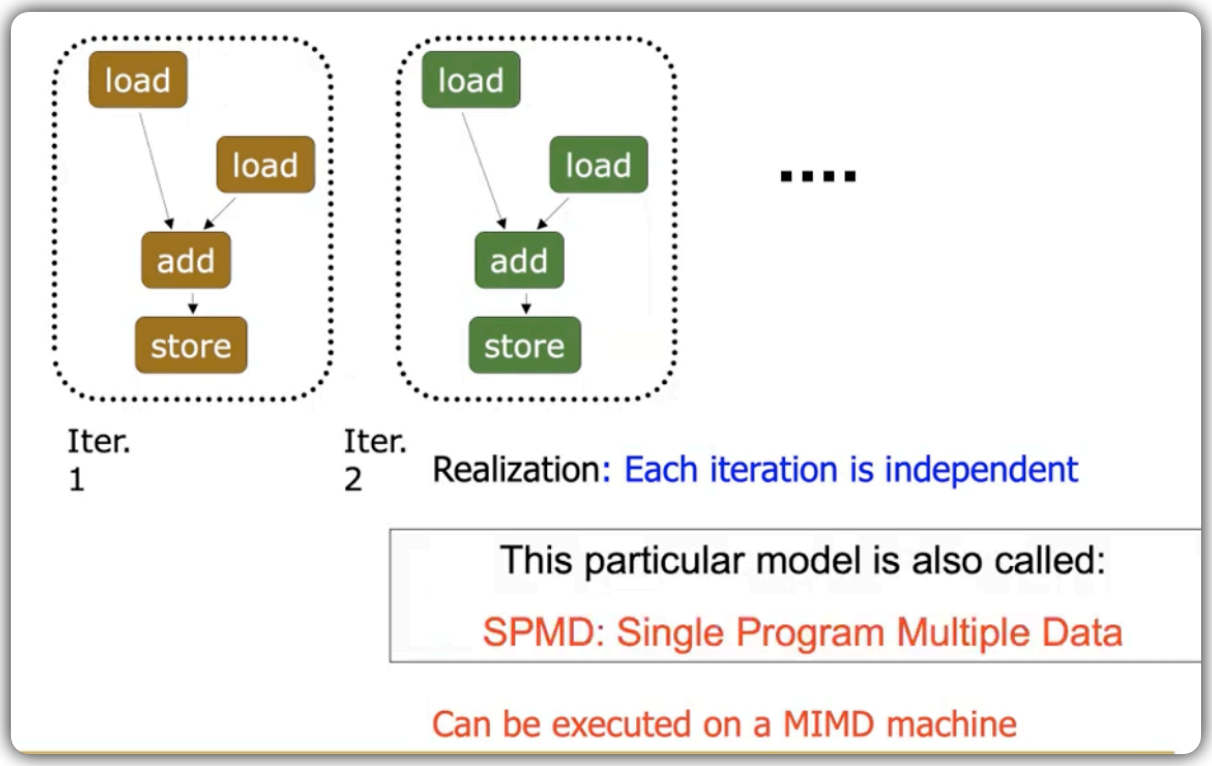
4. 🚀 Understanding Warp-Based SIMD and GPU Execution

🔗

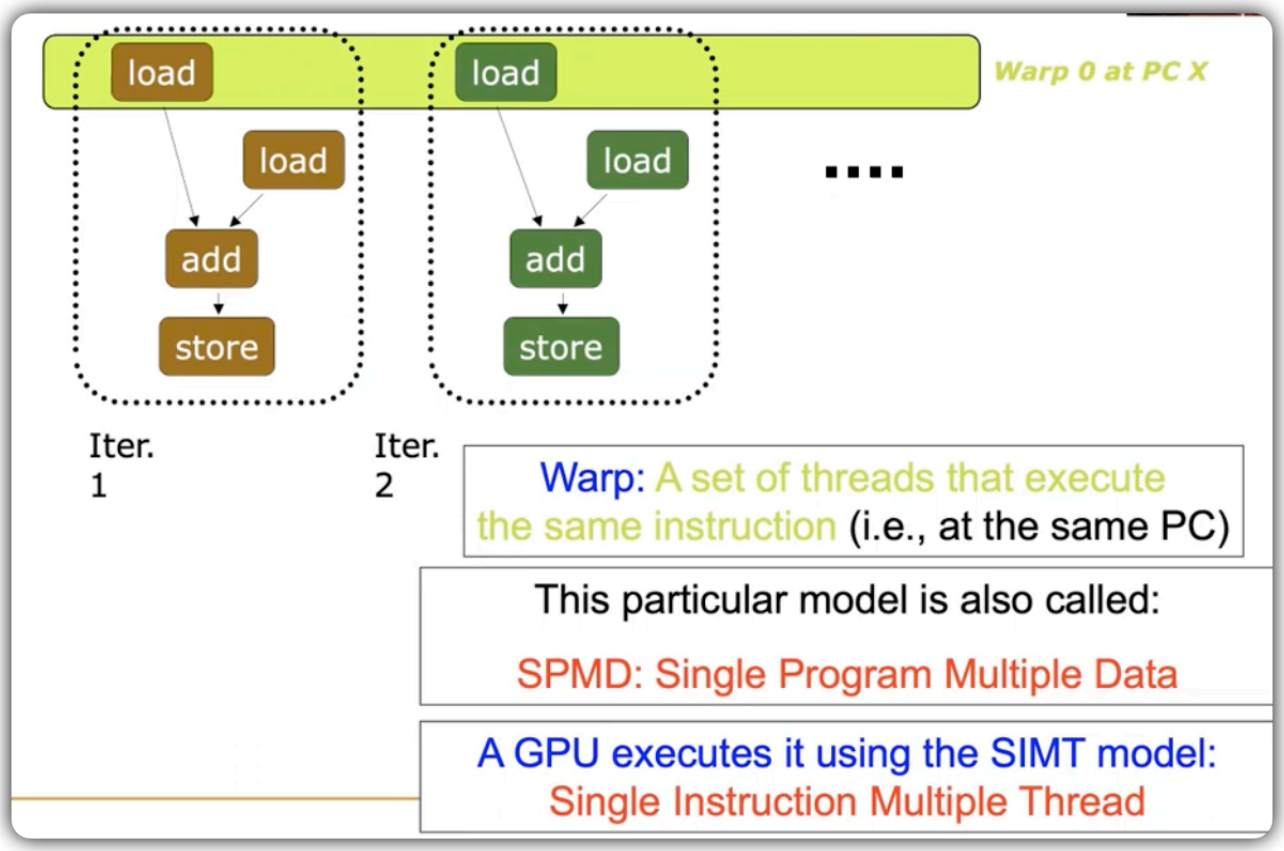
- Threads execute iterations of a **vectorizable loop** on a multi-core CPU or GPU, following the single program multiple data model.
    - **Programming Model**
        - Refers to how the programmer expresses the code
        - Examples:
            - Sequential (von Neumann)
            - Data Parallel (SIMD)
            - Dataflow
            - Multi-threaded (MIMD, SPMD)
    - **Execution Model**
        - Refers to how the hardware executes the code underneath
        - Examples:
            - Out-of-order execution
            - Vector processor
            - Array processor
            - Dataflow processor
            - Multiprocessor
            - Multithreaded processor
    - **Important Note**: Execution Model can be very different from the Programming Model
        - Example: von Neumann model implemented by an OoO processor
        - Example: SPMD model implemented by a SIMD processor (a GPU)
- In GPUs , threads operate based on their ID on different data elements, using a **SPMD** programming model with individual thread context.
    - **data parallel(SIMD)**



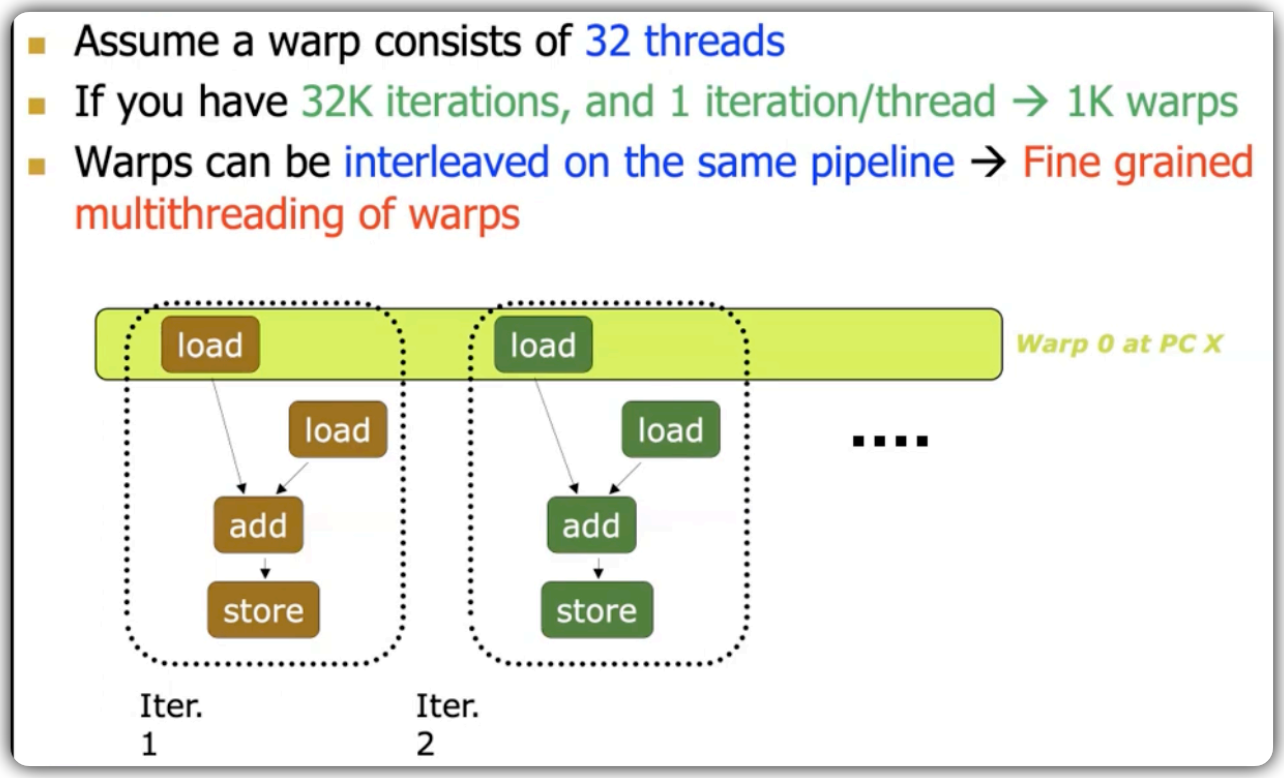
- **multi-thread(SPMD)**



- **Hardware forms threads into a "warp" grouping them for execution as a SIMD instruction.**



- GPUs employ fine-grain multi-threading, **allowing warps to execute instructions in an interleaved manner on the same pipeline.**
    - **SIMT (Single Instruction Multiple Thread)**
        - Multiple instruction streams of scalar instructions
        - Threads grouped dynamically into warps
        - Instructions example:
            - [LD, LD, ADD, ST]
            - NumThreads
    - **Two Major SIMT Advantages**
        - Can treat each thread separately
        - Can execute each thread independently (on any type of scalar pipeline)
        - Enables **MIMD** processing
- Warp-based SIMD uses multiple scalar threads that are later combined into SIMD instructions, differing from traditional SIMD's single thread model.


Assume a warp consists of 32 threads
If you have 32K iterations, and 1 iteration/thread → 1K warps
Warp interleaved on the same pipeline → Fine grained multithreading of warps
load
load
add
store
load
load
add
store
Warp 0 at PC X
Iter. 1
Iter. 2

- example

What is a Warp in a GPU?

In GPU architecture, **a warp** is the smallest execution unit of threads that a GPU scheduler manages. Specifically:

- A **warp** typically consists of **32 threads** in NVIDIA GPUs (though this may vary in other architectures).
- All threads in a warp execute the same instruction (SIMD - Single Instruction Multiple Data), but on different data (data-parallel execution).
- Warps are scheduled by the GPU's **warp scheduler**, which interleaves their execution to hide memory latency and ensure high throughput.

Key Characteristics of a Warp:

1. **SIMD Nature:** All threads execute the same instruction at the same time.
    - If threads in a warp diverge (e.g., take different branches of an `if` statement), the GPU serializes the execution of each branch (called **warp divergence**), reducing efficiency.
2. **Shared Resources:** Threads in a warp share execution resources, such as registers and shared memory.

What is Fine-Grained Multi-Threading?

Fine-grained multi-threading refers to the GPU's ability to switch between different warps at every clock cycle, ensuring continuous utilization of computational resources. It allows the GPU to:

1. Hide **latency** from memory accesses (e.g., waiting for data from global memory).
2. Keep the processing cores busy by interleaving instructions from different warps.

Why is Fine-Grained Multi-Threading Important?

- In GPUs, accessing global memory can take hundreds of clock cycles.
- By switching to another warp while one warp is waiting for memory, the GPU avoids idle cores and achieves high throughput.

Real-World Example: Matrix Multiplication

Consider a GPU performing **matrix multiplication** (e.g., multiplying two 1000×1000 matrices).

Warp Example:

1. **Setup:**

- Each thread in a warp computes a single element of the result matrix.
- For a warp of 32 threads, each thread calculates one output value in the result matrix.

2. **Execution:**
   - Suppose the warp operates on rows 0 and 1 of the matrix.
   - The 32 threads in the warp execute the same instruction (e.g., multiplying corresponding elements in a row and column) but operate on different data points (e.g., elements of the row).

Fine-Grained Multi-Threading Example:

1. **Memory Access:**
   - The threads in the warp need to load values from global memory.
   - Accessing global memory may take 300-600 cycles.
2. **Hiding Latency:**
   - While one warp is waiting for the memory operation, the GPU scheduler switches to another warp ready for computation.
   - For example, while warp 1 is waiting for memory, warp 2 might calculate a partial sum, and warp 3 might perform a different task entirely.
3. **Efficiency:**
   - By interleaving the execution of multiple warps, the GPU ensures that the computational resources remain busy and the memory latency is effectively hidden.

Summary

- **Warp:** A group of threads (usually 32) executing the same instruction in parallel.
- **Fine-Grained Multi-Threading:** A mechanism to interleave warp execution to hide latency and maximize GPU core utilization.
- **Real-World Relevance:** These concepts make GPUs extremely efficient for tasks like deep learning, image processing, and numerical simulations, where data-parallel computations dominate.

- comparsion

Feature	SIMD (Single Instruction Multiple Data)	SIMT (Single Instruction Multiple Threads)
Definition	A single instruction operates on multiple data elements using	A single instruction operates on multiple threads, each with its own

Feature	SIMD (Single Instruction Multiple Data)	SIMT (Single Instruction Multiple Threads)
	vector units.	data.
Execution Model	Vectorized execution: works on data arrays simultaneously.	Thread-based execution: works on independent threads in parallel.
Granularity	Operates on data elements in a vector.	Operates on threads in a warp or block.
Independence	No thread-level independence; operates on grouped data in lockstep.	Threads can diverge (e.g., due to branching), but divergence impacts performance.
Flexibility	Limited flexibility; all lanes must process the same operation on data.	Greater flexibility; threads can access different memory locations and execute independently (to an extent).
Resource Usage	Efficient for uniform operations on large datasets.	Efficient for workloads with high data-level and thread-level parallelism.
Memory Access	Operates on contiguous or strided data arrays (structured).	Threads can access memory independently (structured or unstructured).
Latency Hiding	Typically lacks latency-hiding capabilities.	Can hide latency by interleaving execution of multiple warps.
Usage Contexts	Used in vector processors, multimedia processing, or DSPs (e.g., SSE, AVX).	Used in GPU architectures (e.g., NVIDIA, AMD GPUs) for general-purpose parallel computing.
Performance Impact of Divergence	Does not experience divergence issues since all lanes follow the same path.	Warp divergence can reduce performance when threads in a warp execute different paths.
Hardware Example	Intel SSE, AVX; ARM NEON.	NVIDIA CUDA, AMD GPUs using Compute Units.
Programming Model	Vectorized operations (e.g., using vector libraries or intrinsic functions).	Thread-based programming models (e.g., CUDA, OpenCL).

5. 🚀 Understanding SPMD and SIMD in GPU Programming

🔗

- **SPMD** is a programming model where each processing element executes the same procedure, called threads, that can synchronize at certain points using barriers.
- Modern GPU programming involves multiple threads executing the same program, optimizing execution parallelism and efficiency in SIMD hardware.